Document Specifications
Author: Samyok Poudyal
Date: 9/16/2026
Department: QA
Email: samyogpoudel.14zt@gmail.com

Automation Testing via Selenium Python

---

1. Overview

1.1 Purpose

Automation testing is a software testing approach that uses tools and scripts to automatically execute test cases, validate application functionality, and compare actual results with expected outcomes. Selenium is one of the most widely used open-source automation testing frameworks for web applications, and Python is a popular programming language for writing Selenium test scripts due to its simplicity and readability. Selenium enables testers to automate browser interactions such as navigating web pages, locating web elements, performing user actions, and validating application behavior across different browsers. This report covers the fundamentals of Selenium WebDriver, browser automation, element identification using various locators, synchronization techniques

1.2 Business Objectives

1. Improve software quality by ensuring web applications function as expected.
2. Reduce manual testing effort through automated test execution.
3. Detects defects early by automating repetitive and regression test cases.
4. Increase testing efficiency, accuracy, and consistency across different browsers.
5. Support faster software releases by enabling continuous and reliable testing.
6. Enhance test coverage by automating multiple user workflows and scenarios.

2. Functional Requirements

1. The system should allow users to automate web browsers using Selenium WebDriver.
2. The system should allow users to locate and interact with web elements using different locator strategies.
3. The system should allow users to perform actions such as clicking, typing, selecting dropdowns, and submitting forms.
4. The system should allow users to validate web application behavior by verifying page elements, titles, URLs, and displayed content.
5. The system should support handling of waits, alerts, frames, windows, and browser navigation during test execution.

3. Technical Architecture

Folder Path / Directory:

- AutomationTesting C:\Users\Dell\PycharmProjects\Automati

Files & Folders:

- .venv
- capturingscreenshot.py
- commands.py
- cookies.py
- downloadingfileinedge.py
- Downloadingfiles.py
- downloadingfilesinfirefox.py
- finding_elements.py
- handling alerts.py
- handling table.py
- main.py
- mouseoperations.py
- performing keyboard actions.py
- Scrolling pages.py
- wait commands.py
- wirkingwithexcel.py
- workingwithframes.py
- workingwithsql.py

4. Tools and Environment Used

The automation testing activities were performed using Selenium WebDriver with Python as the primary automation framework. Selenium is an open-source tool used for automating web browsers and validating web application functionality. It allows testers to automate user interactions such as opening web pages, locating web elements, entering data, clicking buttons, and verifying expected outcomes.

The project was implemented using Selenium Python version 4.51.0, which provides the latest WebDriver features, improved browser automation support, and enhanced interaction capabilities with modern web applications. The automation scripts were developed and executed in PyCharm IDE using a Python virtual environment (.venv) to manage project dependencies separately.

Environment Details:

- Programming Language: Python
- Automation Framework: Selenium WebDriver 4.51.0
- IDE: PyCharm
- Package Manager: pip
- Environment: Python Virtual Environment (.venv)
- Browser Automation: Google Chrome with ChromeDriver

5. Automation Testing via Selenium Python

1. Basic Selenium WebDriver Commands

Selenium WebDriver provides various commands to automate browser activities and interact with web applications. Basic Selenium commands are used to initialize the browser, navigate to web pages, retrieve page information, locate web elements, perform user actions, and close the browser session. These commands form the foundation of Selenium automation and are commonly used while creating automated test scripts.

Explanation of Commands

a. Creating WebDriver Instance
   webdriver.Chrome() is used to create an instance of Chrome WebDriver and launch the Chrome browser.

b. Opening Web Page
   driver.get() navigates the browser to the specified URL.

c. Maximizing Browser Window
   driver.maximize_window() maximizes the browser window to ensure better visibility and interaction with web elements.

d. Getting Page Title
   driver.title retrieves the title of the current web page.

e. Getting Current URL
   driver.current_url returns the URL of the currently opened webpage.

f. Finding Web Element
   driver.find_element() is used to locate elements on the webpage.

   The By.XPATH locator is used to identify the input field using its XPath expression.
   Clearing and Entering Data

   clear() removes any existing text from the input field.
   send_keys() enters the specified text into the selected input field.

g. Closing Browser Session
   driver.quit() closes all browser windows and terminates the WebDriver session.

Attachments:

basic-selenium-commands.png

2. Application, Conditional, Browser and Navigational Commands

Selenium WebDriver provides different types of commands to interact with web applications and control browser behavior during automation testing. These commands help testers perform actions such as opening and closing browsers, navigating between web pages, refreshing pages, handling browser history, and applying conditional checks to validate the presence, visibility, and state of web elements.

Application commands are used to retrieve information about the current web page, browser commands control browser-level actions, navigation commands manage movement between different URLs, and conditional commands validate web element conditions during test execution.

2.1 Explanation of Commands

Application Commands

a. Opening Webpage

driver.get() is used to open a specific webpage URL in the browser.

b. Getting Page Title

driver.title returns the title of the currently opened webpage.

c. Getting Current URL

driver.current_url returns the URL of the currently opened webpage.

d. Finding Web Element

driver.find_element() is used to locate a specific web element on the webpage using different locator strategies such as ID, Name, XPath, CSS Selector, and others.

Browser Commands

a. Maximizing Browser Window

driver.maximize_window() is used to maximize the browser window during test execution for better visibility of web elements.

b. Getting Browser Window Size

driver.get_window_size() returns the current size of the browser window.

c. Closing Current Browser Window

driver.close() is used to close the currently active browser window.

d. Closing All Browser Windows

driver.quit() closes all browser windows and terminates the WebDriver session.

Navigational Commands

a. Refreshing Webpage

driver.refresh() reloads the current webpage.

b. Navigating Backward

driver.back() navigates the browser to the previous webpage in the browser history.

c. Navigating Forward

driver.forward() navigates the browser to the next webpage in the browser history.

d. Navigating to Another URL

driver.get() is used to navigate directly to a specified URL.

Conditional Commands

a. Checking Element Visibility

is_displayed() checks whether a web element is visible on the webpage. It returns True if the element is displayed.

b. Checking Element Availability

is_enabled() checks whether a web element is enabled and available for user interaction.

c. Checking Selection Status

is_selected() checks whether a checkbox or radio button is selected. It returns True if the element is selected.

Attachments:

navigational, browser, conditional, application commands.png

3. Finding Web Elements

driver.find_element() is used to locate a single web element on a webpage. It requires a locator strategy to identify the element, such as ID, Name, Class Name, XPath, CSS Selector, Link Text, or Tag Name.

3.1 Common Locator Methods:

a. Finding Element by ID

   By.ID is used to locate an element using its unique ID attribute.

b. Finding Element by Name

   By.NAME is used to locate an element using its name attribute.

c. Finding Element by Class Name

   By.CLASS_NAME is used to locate an element using its class attribute.

d. Finding Element by XPath

   By.XPATH is used to locate elements using XPath expressions.

e. Finding Element by CSS Selector

   By.CSS_SELECTOR is used to locate elements using CSS selector patterns.

f. Finding Element by Link Text

   By.LINK_TEXT is used to locate a link using its complete visible text.

g. Finding Element by Tag Name

   By.TAG_NAME is used to locate elements using HTML tag names.

h. Finding Multiple Elements

   driver.find_elements() is used to locate multiple web elements that match the given locator.

Attachments:

findingelements.png , findingelements1.png , findingelements2.png

4. Wait Commands

Selenium wait commands are used to handle synchronization issues between the Selenium script and the web application. They allow the script to wait until an element or condition is ready before performing an action.

4.1 Common Wait Commands

a. Implicit Wait

   driver.implicitly_wait() is used to tell Selenium to wait for a specific amount of time while locating web elements. It applies globally to all elements in the WebDriver session.

   Syntax:

   ```
   driver.implicitly_wait(time_in_seconds)
   ```

   Example:

   ```
   driver.implicitly_wait(10)element = driver.find_element(By.ID, "input-search")element.send_keys("Laptop")
   ```

b. Explicit Wait

   WebDriverWait() is used to wait for a specific condition to occur before continuing the execution. It works with expected conditions.

Syntax:

```
WebDriverWait(driver, timeout).until(condition)
```

Example:

```
wait = WebDriverWait(driver, 10)

element = wait.until(

    EC.presence_of_element_located((By.ID, "input-search"))

) element.send_keys("Laptop")
```

c. Presence of Element Located

presence_of_element_located() is used to wait until an element is present in the HTML DOM. The element does not necessarily need to be visible.

Syntax:

```
EC.presence_of_element_located((By.LOCATOR, "value"))
```

Example:

```
element = WebDriverWait(driver, 10).until(
EC.presence_of_element_located((By.ID, "input-search")))
```

d. Visibility of Element Located

visibility_of_element_located() is used to wait until an element is present and visible on the webpage.

Syntax:

```
EC.visibility_of_element_located((By.LOCATOR, "value"))
```

Example:

```
element = WebDriverWait(driver, 10).until( EC.visibility_of_element_located((By.ID, "input-search")))
```

Attachments:

waitcommand.png waitcommand1.png waitcommand2.png

, waitcommand3.png waitcommand4.png

5. Handling Alerts

Selenium alerts are JavaScript pop-up messages that require user interaction before continuing further actions. Selenium uses driver.switch_to.alert to switch the control from the webpage to the alert window.

5.1 Common Alert Commands

a. Switching to Alert

driver.switch_to.alert is used to switch Selenium control from the webpage to the JavaScript alert popup.

Syntax:

```
driver.switch_to.alert
```

Example:

```
alert = driver.switch_to.alert
```

b. Accepting Alert

alert.accept() is used to click the OK button on an alert popup.

Syntax:

```
alert.accept()
```

Example:

```
alert = driver.switch_to.alertalert.accept()
```

c. Dismissing Alert

alert.dismiss() is used to click the Cancel button on a confirmation alert.

Syntax:

```
alert.dismiss()
```

Example:

```
alert = driver.switch_to.alertalert.dismiss()
```

d. Getting Alert Text

alert.text is used to retrieve the message displayed inside the alert popup.

Syntax:

```
alert.text
```

Example:

```
alert = driver.switch_to.alertmessage = alert.textprint(message)
```

e. Sending Text to Prompt Alert

alert.send_keys() is used to enter text into a prompt alert input box.

Syntax:

```
alert.send_keys("text")
```

Example:

```
alert = driver.switch_to.alert

alert.send_keys("Selenium")

alert.accept()
```

Attachments:

handling tables.png , handlingalert1.png , handlilngalerts2.png

6. Working with Frames

Frames (or iframes) are HTML documents embedded inside another HTML document. Selenium cannot directly access elements inside a frame until the driver switches its focus to that frame.

Selenium provides driver.switch_to.frame() to switch from the main webpage to a frame and driver.switch_to.default_content() to return back to the main page.

6.1 Common Frame Commands

a. Switching to Frame by Index

driver.switch_to.frame(index) is used to switch to a frame using its index number.

Syntax:

```
driver.switch_to.frame(index)
```

Example:

```
driver.switch_to.frame(0)
```

b. Switching to Frame by Name or ID

driver.switch_to.frame() is used to switch to a frame using its name or ID attribute.

Syntax:

```
driver.switch_to.frame("frame_name")
```

Example:

```
driver.switch_to.frame("frame1")
```

c. Switching to Frame using Web Element

driver.switch_to.frame() is used to switch to a frame using a located frame element.

Syntax:

driver.switch_to.frame(frame_element)

Example:

```
frame = driver.find_element(By.ID, "frame1")driver.switch_to.frame(frame)
```

d. Switching Back to Main Page

driver.switch_to.default_content() is used to switch Selenium control back to the main webpage from the frame.

Syntax:

```
driver.switch_to.default_content()
```

Example:

```
driver.switch_to.default_content()
```

e. Switching to Parent Frame

driver.switch_to.parent_frame() is used to switch Selenium control from the current frame to its parent frame.

Syntax:

```
driver.switch_to.parent_frame()
```

Example:

```
driver.switch_to.parent_frame()
```

Attachments:
workingwithframes.png , workingwithframes1.png

7. Handling Web Tables

Web tables are HTML structures used to display data in rows and columns. In Selenium, web tables are handled by locating the table elements (table, tr, th, td) and extracting or interacting with the data.

Selenium does not provide a direct table handling method. We use locators like XPath, CSS Selector, and loops to read and interact with table data.

7.1 Common Web Table Operations

a. Finding a Web Table

find_element() is used to locate a table on the webpage.

Syntax:

```
driver.find_element(By.LOCATOR, "value")
```

Example:

```
table = driver.find_element(By.TAG_NAME, "table")
```

b. Reading Table Headers

find_elements() is used to locate all header columns (th) in a table.

Syntax:

```
driver.find_elements(By.TAG_NAME, "th")
```

Example:

```
headers = driver.find_elements(By.TAG_NAME, "th")for header in headers:
print(header.text)
```

c. Reading All Rows from a Table

find_elements() is used to locate all table rows (tr).

Syntax:

```
driver.find_elements(By.TAG_NAME, "tr")
```

Example:

```
rows = driver.find_elements(By.TAG_NAME, "tr")
for row in rows:
    print(row.text)
```

d. Reading Data from Table Cells

find_elements() is used to locate table data cells (td) inside rows.

Syntax**:**

```
row.find_elements(By.TAG_NAME, "td")
```

Example:

```
rows = driver.find_elements(By.TAG_NAME, "tr")
for row in rows:
    columns = row.find_elements(By.TAG_NAME, "td")
    for column in columns:
        print(column.text)
```

e. Finding Specific Data in a Table

XPath is used to locate a specific row or cell based on table data.

Syntax:

```
//tag[text()='value']
```

Example:

```
element = driver.find_element( By.XPATH,    "//td[text()='John']")
print(element.text)
```

f. Counting Number of Rows

len() is used to find the total number of rows in a table.

Example:

```
rows = driver.find_elements(By.XPATH, "//table/tbody/tr")
print("Total Rows:", len(rows))
```

g. Counting Number of Columns

len() is used to find the total number of columns in a table.

Example:

```
columns = driver.find_elements(By.XPATH, "//table/thead/tr/th")
print("Total Columns:", len(columns))
```

Attachments:

handling tables.png , handlingtables.png

8. Mouse Operations

Mouse operations in Selenium are performed using the ActionChains class. It is used to handle advanced mouse actions such as hovering, clicking, double-clicking, right-clicking, and drag-and-drop operations.

ActionChains allows Selenium to simulate real user mouse interactions.

8.1 Common Mouse Operations

a. Mouse Hover (Move To Element)

move_to_element() is used to move the mouse pointer over a specific element.

Syntax:

```
actions.move_to_element(element).perform()
```

Example:

```
 menu = driver.find_element(By.LINK_TEXT,
"Components")actions.move_to_element(menu).perform()
```

b. Left Click

click() is used to perform a normal left mouse click.

Syntax:

```
actions.click(element).perform()
```

Example:

```
button = driver.find_element(By.ID, "button")actions.click(button).perform()
```

c. Double Click

double_click() is used to perform a double mouse click on an element.

Syntax:

```
actions.double_click(element).perform()
```

Example:

```
element = driver.find_element(By.ID,
"doubleClick")actions.double_click(element).perform()
```

d. Right Click (Context Click)

context_click() is used to perform a right mouse click.

Syntax:

```
actions.context_click(element).perform()
```

Example:

```
 element = driver.find_element(By.ID,
"rightClick")actions.context_click(element).perform()
```

e. Drag and Drop

drag_and_drop() is used to drag an element from one location and drop it onto another element.

Syntax:

```
actions.drag_and_drop(source, target).perform()
```

Example:

```
source = driver.find_element(By.ID, "source")target = driver.find_element(By.ID,
"target")actions.drag_and_drop(source, target).perform()
```

f. Click and Hold

click_and_hold() is used to click and hold an element without releasing the mouse button.

Syntax:

```
actions.click_and_hold(element).perform()
```

Example:

```
element = driver.find_element(By.ID,
"box")actions.click_and_hold(element).perform()
```

g. Release Mouse Button

release() is used to release the mouse button after clicking and holding.

Syntax:

```
actions.release().perform()
```

Example:

```
actions.release().perform()
```

h. Move By Offset

move_by_offset() moves the mouse pointer from its current position by given X and Y coordinates.

Syntax:

```
actions.move_by_offset(x, y).perform()
```

Example:

```
actions.move_by_offset(100, 50).perform()
```

Attachments:

mouseoperations.png mouseoperation1.png mouseoperations2.png

mouseoprations3.png

9. Scrolling Web Pages

Scrolling is used to move up or down on a webpage to access elements that are not currently visible. Selenium performs scrolling using JavaScript through the execute_script() method.

9.1 Common Scrolling Commands

a. **Scrolling Down by Pixel**

window.scrollBy() is used to scroll the webpage by a specified number of pixels.

Syntax:

```
driver.execute_script("window.scrollBy(x, y)")
```

Example:

```
driver.execute_script("window.scrollBy(0,500)")
```

b. **Scrolling Up by Pixel**

window.scrollBy() is used to scroll the webpage upward by specifying a negative Y value.

Syntax:

```
driver.execute_script("window.scrollBy(x, -y)")
```

Example:

```
driver.execute_script("window.scrollBy(0,-500)")
```

c. Scrolling to the Bottom of the Page

document.body.scrollHeight is used to scroll directly to the bottom of the webpage.

Syntax:

```
driver.execute_script("window.scrollTo(0, document.body.scrollHeight)")
```

Example:

```
driver.execute_script("window.scrollTo(0, document.body.scrollHeight)")
```

d. Scrolling to the Top of the Page

window.scrollTo() is used to scroll directly to the top of the webpage.

Syntax:

```
driver.execute_script("window.scrollTo(0, 0)")
```

Example:

```
driver.execute_script("window.scrollTo(0,0)")
```

e. Scrolling Until an Element is Visible

scrollIntoView() is used to scroll the page until a specific web element becomes visible.

Syntax:

```
driver.execute_script("arguments[0].scrollIntoView();", element)
```

Example:

```
element = driver.find_element(By.LINK_TEXT, "About
Us")driver.execute_script("arguments[0].scrollIntoView();", element)
```

f. Horizontal Scrolling

window.scrollBy() is used to scroll the webpage horizontally.

Syntax:

```
driver.execute_script("window.scrollBy(x, 0)")
```

Example:

```
driver.execute_script("window.scrollBy(500,0)")
```

Attachments:

scrollingpages.png , scrollingpages1.png

10. Performing Keyboard Actions

Keyboard actions in Selenium are performed using the Keys class. It allows automation scripts to simulate keyboard inputs such as pressing Enter, Tab, Escape, Ctrl+A, Ctrl+C, Ctrl+V, function keys, and other keyboard shortcuts.

The Keys class is available in the selenium.webdriver.common.keys module.

10.1 Common Keyboard Actions

Importing Keys

Keys are used to simulate keyboard key presses.

Syntax:

```
from selenium.webdriver.common.keys import Keys
```

Example:

```
from selenium.webdriver.common.keys import Keys
```

a. Pressing Enter Key

Keys.ENTER is used to simulate pressing the Enter key.

Syntax:

```
element.send_keys(Keys.ENTER)
```

Example:

```
search_box = driver.find_element(By.NAME,
"search")search_box.send_keys("Laptop")search_box.send_keys(Keys.ENTER)
```

b. Pressing Tab Key

Keys.TAB is used to move the cursor to the next input field.

Syntax:

element.send_keys(Keys.TAB)

Example:

username = driver.find_element(By.ID,"input-email")username.send_keys(Keys.TAB)

c. Pressing Escape Key

Keys.ESCAPE is used to simulate pressing the Escape key.

Syntax:

element.send_keys(Keys.ESCAPE)

Example:

driver.find_element(By.TAG_NAME, "body").send_keys(Keys.ESCAPE)

d. Selecting All Text (Ctrl + A)

Keys.CONTROL + "a" is used to select all text inside an input field.

Syntax:

element.send_keys(Keys.CONTROL, "a")

Example:

search_box.send_keys(Keys.CONTROL, "a")

e. Copying Text (Ctrl + C)

Keys.CONTROL + "c" is used to copy selected text.

Syntax:

element.send_keys(Keys.CONTROL, "c")

Example:

```
search_box.send_keys(Keys.CONTROL, "c")
```

f. Pasting Text (Ctrl + V)

Keys.CONTROL + "v" is used to paste copied text.

Syntax:

```
element.send_keys(Keys.CONTROL, "v")
```

Example:

```
search_box.send_keys(Keys.CONTROL, "v")
```

g. Cutting Text (Ctrl + X)

Keys.CONTROL + "x" is used to cut selected text.

Syntax:

```
element.send_keys(Keys.CONTROL, "x")
```

Example:

```
search_box.send_keys(Keys.CONTROL, "x")
```

h. Deleting Text

Keys.DELETE is used to delete characters after the cursor.

Syntax:

```
element.send_keys(Keys.DELETE)
```

Example:

```
search_box.send_keys(Keys.DELETE)
```

i. Using Backspace

Keys.BACKSPACE is used to delete characters before the cursor.

Syntax:

```
element.send_keys(Keys.BACKSPACE)
```

Example:

```
search_box.send_keys(Keys.BACKSPACE)
```

Attachments:
performingkeyboardaction.png , performingkeyboardactions1.png , performingkeyboardaction2.png

11. Downloading a File

Downloading files in Selenium involves triggering the download action and configuring the browser to automatically save files to a specified location without displaying the download dialog.

Browser preferences (especially in Chrome) are commonly used to control the download behavior.

11.1 Common File Download Operations

a. Setting Download Directory

ChromeOptions() is used to configure the default download folder.

Syntax:

options = webdriver.ChromeOptions()options.add_experimental_option("prefs", preferences)

Example:

```
download_path = "C:\\Downloads"prefs = {    "download.default_directory": download_path}
options = webdriver.ChromeOptions()options.add_experimental_option("prefs", prefs)
driver = webdriver.Chrome(options=options)
```

b. Downloading a File

click() is used to start downloading a file by clicking the download link or button.

Syntax:

element.click()

Example:

download_button=driver.find_element(By.LINK_TEXT,"Download")download_button.click()

c. Verifying File Download

The os.path.exists() method is used to verify whether the downloaded file exists in the download directory.

Syntax:

os.path.exists(file_path)

Example:

```
import osfile_path = "C:\\Downloads\\sample.pdf"if os.path.exists(file_path): print("Download Successful")else:    print("Download Failed")
```

Attachments:

downloadingfile.png , downloadingfile1.png

11.2 Downloading Files in Firefox

Firefox allows automatic file downloads by configuring browser preferences using FirefoxOptions(). These preferences specify the download directory and disable the download confirmation dialog.

Configuring Firefox Download Directory

FirefoxOptions() is used to configure the default download folder and automatically save downloaded files.

Syntax:

options = webdriver.FirefoxOptions()options.set_preference("browser.download.dir", download_path)

Attachments:

downloadingfilesinfirefox.png , downloadingfielsinfirefox1.png

11.3 Downloading Files in Microsoft Edge

Microsoft Edge uses the Chromium engine, so downloading files is configured similarly to Google Chrome using EdgeOptions().

Configuring Edge Download Directory

EdgeOptions() is used to configure the default download folder and disable the download prompt.

Syntax:

options = webdriver.EdgeOptions()options.add_experimental_option("prefs", preferences)

Attachments:

downloadingfileinedge.png , downloadingfileinedge1.png

12. Getting Screenshots

Screenshots are used to capture the current state of a webpage during test execution. They are helpful for debugging, reporting test failures, and documenting test results.

Selenium provides methods to capture screenshots of the entire webpage or a specific web

element.

12.1 Common Screenshot Commands

a. Capturing Full Page Screenshot

driver.save_screenshot() is used to capture and save a screenshot of the current webpage.

Syntax:

driver.save_screenshot("file_path")

Example:

driver.save_screenshot("homepage.png")

b. Capturing Screenshot using get_screenshot_as_file()

get_screenshot_as_file() is used to capture the current webpage and save it to a specified file.

Syntax:

driver.get_screenshot_as_file("file_path")

Example:

driver.get_screenshot_as_file("login_page.png")

c. Capturing Screenshot as PNG

get_screenshot_as_png() captures the screenshot as PNG image data.

Syntax:

driver.get_screenshot_as_png()

Example:

png_data = driver.get_screenshot_as_png()

d. Capturing Screenshot as Base64

get_screenshot_as_base64() captures the screenshot in Base64 format.

Syntax:

driver.get_screenshot_as_base64()

Example:

base64_data = driver.get_screenshot_as_base64()

e. Capturing Screenshot of a Web Element

element.screenshot() is used to capture a screenshot of a specific web element.

Syntax:

element.screenshot("file_path")

Example:

logo = driver.find_element(By.CLASS_NAME,"img-responsive")logo.screenshot("logo.png")

Attachments: screeenshot.png , screenshot1.png

13. Handling Cookies

Cookies are small pieces of data stored by the browser to remember user information such as login sessions, preferences, and tracking information. Selenium provides methods to add, retrieve, delete, and manage browser cookies during automation.

13.1 Common Cookie Commands

a. Getting All Cookies

driver.get_cookies() is used to retrieve all cookies stored by the current webpage.

Syntax:

driver.get_cookies()

Example:

cookies = driver.get_cookies()print(cookies)

b. Getting a Specific Cookie

driver.get_cookie() is used to retrieve a specific cookie by its name.

Syntax:

driver.get_cookie("cookie_name")

Example:

cookie = driver.get_cookie("PHPSESSID")print(cookie)

c. Adding a Cookie

driver.add_cookie() is used to add a new cookie to the current browser session.

Syntax:

driver.add_cookie({"name":"cookie_name","value":"cookie_value"})

Example:

driver.add_cookie({ "name": "username", "value": "selenium"})

d. Deleting a Specific Cookie

driver.delete_cookie() is used to delete a cookie by its name.

Syntax:

driver.delete_cookie("cookie_name")

Example:

driver.delete_cookie("username")

e. Deleting All Cookies

driver.delete_all_cookies() is used to remove all cookies from the current browser session.

Syntax:

driver.delete_all_cookies()

Example:

driver.delete_all_cookies()

Attachments:

cookies.png , cookies1.png

14. Working with Excel

Excel files are commonly used in Selenium for data-driven testing, where test data is stored in spreadsheets instead of hardcoding values in the script. The openpyxl library is widely used to read from and write to Excel (.xlsx) files.

14.1 Common Excel Operations

a. Loading an Excel Workbook

load_workbook() is used to open an existing Excel workbook.

Syntax:

load_workbook("file_path")

Example:

```
from openpyxl import load_workbook
workbook = load_workbook("TestData.xlsx")
```

b. Selecting a Worksheet

A worksheet is selected by its name.

Syntax:

```
workbook["SheetName"]
```

Example:

```
sheet = workbook["Sheet1"]
```

c. Reading Data from a Cell

cell() is used to read data from a specific row and column.

Syntax:

```
sheet.cell(row, column).value
```

Example:

```
username = sheet.cell(2, 1).valueprint(username)
```

d. Writing Data to a Cell

cell() is also used to write data into a specific cell.

Syntax:

```
sheet.cell(row, column).value = data
```

Example:

```
sheet.cell(2, 3).value = "Passed"
```

e. Saving the Workbook

save() is used to save changes made to the Excel workbook.

Syntax:

```
workbook.save("file_path")
```

Example:

```
workbook.save("TestData.xlsx")
```

f. Getting Total Rows

max_row returns the total number of rows in a worksheet.

Syntax:

```
sheet.max_row
```

Example:

```
rows = sheet.max_rowprint(rows)
```

g. Getting Total Columns

max_column returns the total number of columns in a worksheet.

Syntax:

```
sheet.max_column
```

Example:

```
columns = sheet.max_columnprint(columns)
```

Attachments:

workingwithexvcel.png , workingwithexcel1.png

15. SQL with Selenium

SQL (Structured Query Language) is used to interact with databases. In Selenium automation, SQL is commonly used to verify test data, retrieve data, insert records, update data, or delete records during data-driven testing.

Python commonly uses the mysql.connector library to connect to a MySQL database.

15.1 Common SQL Operations

a. Importing MySQL Connector

mysql.connector is used to connect Python with a MySQL database.

Syntax:

```
import mysql.connector
```

Example:

```
import mysql.connector
```

b. Connecting to the Database

mysql.connector.connect() is used to establish a connection with the database.

Syntax:

```
mysql.connector.connect(    host="host_name",    user="username",
password="password",    database="database_name")
```

Example:

```
connection = mysql.connector.connect(    host="localhost",    user="root",
password="password",    database="testdb")
```

c. Creating a Cursor

cursor() is used to create a cursor object for executing SQL queries.

Syntax:

connection.cursor()

Example:

cursor = connection.cursor()

d. Executing a SELECT Query

execute() is used to execute an SQL query.

Syntax:

```
cursor.execute("SELECT * FROM table_name")
```

Example:

```
cursor.execute("SELECT * FROM users")
```

e. Fetching All Records

fetchall() retrieves all rows returned by a query.

Syntax:

```
cursor.fetchall()
```

Example:

```
records = cursor.fetchall()for row in records:    print(row)
```

f. Fetching One Record

fetchone() retrieves only the first row returned by a query.

Syntax:

cursor.fetchone()

Example:

record = cursor.fetchone()print(record)

g. Inserting Data

INSERT INTO is used to add new records to a table.

Syntax:

cursor.execute("INSERT INTO table_name VALUES(...)")

Example:

query = "INSERT INTO users(name, age) VALUES(%s, %s)"values = ("John", 25)cursor.execute(query, values)connection.commit()

h. Updating Data

UPDATE is used to modify existing records.

Syntax:

cursor.execute("UPDATE table_name SET column=value")

Example:

query = "UPDATE users SET age=%s WHERE name=%s"values = (26, "John")cursor.execute(query, values)connection.commit()

i. Deleting Data

DELETE is used to remove records from a table.

Syntax:

cursor.execute("DELETE FROM table_name WHERE condition")

Example:

query = "DELETE FROM users WHERE name=%s"values = ("John",)cursor.execute(query, values)connection.commit()

j. Closing Database Connection

close() is used to close the cursor and database connection.

Syntax:

cursor.close()connection.close()

Example:

cursor.close()connection.close()

Attachments:

sql.png , sql1.png

6. Testing Strategy

Selenium Web Driver via Python

7. References

1. https://www.youtube.com/watch?v=2DD-ynCIZ4w&list=PLUDwpEzHYYLsuUBvuoYTlN0KsBB5t-BDa